# UNIVERSIDAD DE GUADALAJARA

**CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS**

Departamento de Innovación Basada en la Información y el Conocimiento




Ingeniería Informática

| | |
|---|---|
| **Nombre**: Aguilar Nova Eric Oswaldo | **Código**: 219635295 |
| **Nombre de la clase: IoT** | **Nombre del profesor**: Mtro. Zeus Cobián |
| **Atributos de la competencia**: ***Experimentación y curiosidad tecnológica*** | **Práctica**: 2 |
| **Fecha**: 29/08/26 | |
| | |



## Resumen

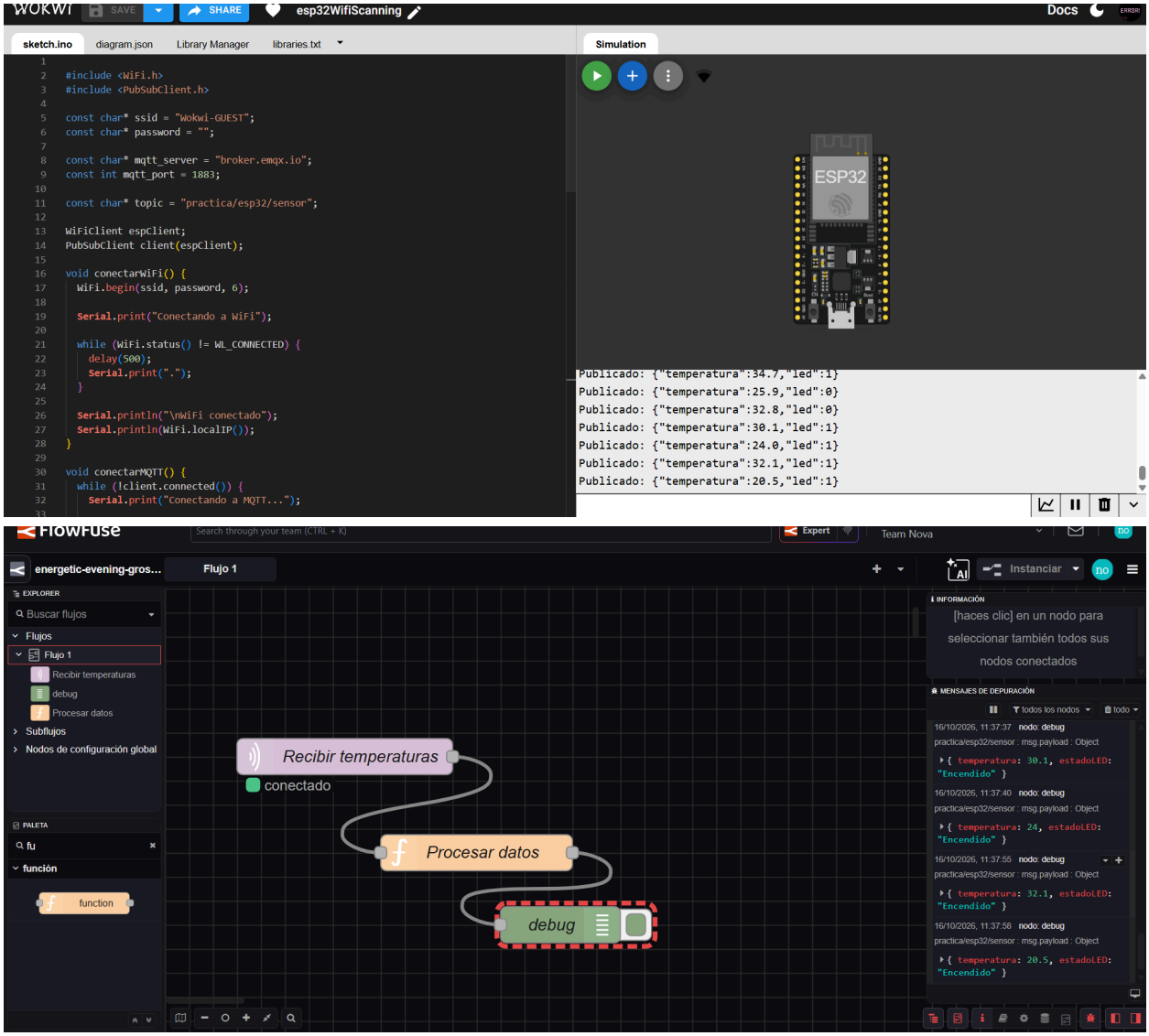
En esta práctica se realizó una simulación de comunicación IoT utilizando un ESP32 en la plataforma Wokwi, el protocolo MQTT y Node-RED. El objetivo fue enviar datos desde un dispositivo simulado hacia una herramienta de visualización y procesamiento, sin necesidad de utilizar componentes electrónicos físicos.

Primero, se creó el proyecto del ESP32 y se programó en el archivo sketch.ino la conexión a la red WiFi Wokwi-GUEST. Posteriormente, se configuró la biblioteca PubSubClient para establecer comunicación con el broker público broker.emqx.io mediante el puerto 1883. El programa generó valores aleatorios de temperatura y estados de un LED, que se enviaron cada tres segundos en formato JSON al tema practica/esp32/sensor.

Después, se utilizó Node-RED desde FlowFuse para recibir los mensajes. Se configuró un nodo MQTT In con el mismo servidor y tema, y se conectó a un nodo Debug para verificar la llegada de los datos. Finalmente, se planteó el uso de un nodo Function para interpretar el mensaje y convertir el estado numérico del LED en los textos «Encendido» y «Apagado».

Como resultado, se comprobó la recepción de temperaturas en Node-RED y se comprendió cómo interactúan un dispositivo IoT, un broker MQTT y una aplicación que procesa información en tiempo real.

# Simulación


Wokwi
SAVE
SHARE
esp32WifiScanning
Docs
sketch.ino
diagram.json
Library Manager
libraries.txt
Simulation
ESP32
Publicado: {"temperatura":34.7,"led":1}
Publicado: {"temperatura":25.9,"led":0}
Publicado: {"temperatura":32.8,"led":0}
Publicado: {"temperatura":30.1,"led":1}
Publicado: {"temperatura":24.0,"led":1}
Publicado: {"temperatura":32.1,"led":1}
Publicado: {"temperatura":20.5,"led":1}
FlowFuse
energetic-evening-gros...
Flujo 1
Instanciar
Recibir temperaturas
conectado
Procesar datos
debug
[haces clic] en un nodo para seleccionar también todos sus nodos conectados
MENSAJES DE DEPURACIÓN
16/10/2026, 11:37:37 nodo: debug
practica/esp32/sensor : msg.payload : Object
{ temperatura: 30.1, estadoLED: "Encendido" }
16/10/2026, 11:37:40 nodo: debug
practica/esp32/sensor : msg.payload : Object
{ temperatura: 24, estadoLED: "Encendido" }
16/10/2026, 11:37:55 nodo: debug
practica/esp32/sensor : msg.payload : Object
{ temperatura: 32.1, estadoLED: "Encendido" }
16/10/2026, 11:37:58 nodo: debug
practica/esp32/sensor : msg.payload : Object
{ temperatura: 20.5, estadoLED: "Encendido" }

## Código Wokwi

```
#include <WiFi.h>
#include <PubSubClient.h>

// Datos de la red WiFi
const char* wifi = "Wokwi-GUEST";
const char* password = "";

// Configuracion del servidor MQTT
const char* servidor = "broker.emqx.io";
const int puerto = 1883;
const char* tema = "practica/esp32/sensor";

// Clientes para la conexion
WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

void conectarWifi() {
  Serial.print("Conectando a WiFi");

  WiFi.begin(wifi, password, 6);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi conectado");
}

void conectarMQTT() {
  while (!mqtt.connected()) {
    Serial.println("Conectando al servidor MQTT...");
```

```
    String id = "ESP32-" + String(random(10000));

    if (mqtt.connect(id.c_str())) {
      Serial.println("MQTT conectado");
    } else {
      Serial.println("Error de conexion MQTT");
      delay(2000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  conectarWifi();

  mqtt.setServer(servidor, puerto);
}

void loop() {
  // Verificar conexion MQTT
  if (!mqtt.connected()) {
    conectarMQTT();
  }

  mqtt.loop();

  // Generar datos simulados
  float temperatura = random(200, 350) / 10.0;
  int led = random(0, 2);
```

```
  // Crear mensaje JSON
  String mensaje = "{";
  mensaje += "\"temperatura\":" + String(temperatura, 1);
  mensaje += ",\"led\":" + String(led);
  mensaje += "}";

  // Enviar datos al broker
  mqtt.publish(tema, mensaje.c_str());

  Serial.println("Datos enviados: " + mensaje);

  delay(3000);
}
```

## Código JSON Node-RED


```
[
    {
        "id": "cd3566b28c08eb81",
        "type": "tab",
        "label": "Flujo 1",
        "disabled": false,
        "info": "",
        "env": []
    },
    {
        "id": "edb379a376c994c6",
        "type": "mqtt in",
        "z": "cd3566b28c08eb81",
        "name": "Recibir temperaturas",
        "topic": "practica/esp32/sensor",
        "qos": "0",
        "datatype": "auto-detect",
        "broker": "8e7cf5f6b06e09aa",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 150,
        "y": 520,
        "wires": [
            [
                "40b7b1b995ee1281"
            ]
        ]
    },
    {
        "id": "8ba7e946245dd1df",
```

```
        "type": "debug",
        "z": "cd3566b28c08eb81",
        "name": "debug",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 330,
        "y": 660,
        "wires": []
    },
    {
        "id": "40b7b1b995ee1281",
        "type": "function",
        "z": "cd3566b28c08eb81",
        "name": "Procesar datos",
        "func": "\nlet datos = msg.payload;\n\n// Convertir el
mensaje a un objeto\nif (typeof datos === \"string\") {\n
datos = JSON.parse(datos);\n}\n\n// Preparar la
información\nmsg.payload = {\n    temperatura:
datos.temperatura,\n    estadoLED: datos.led === 1 ?
\"Encendido\" : \"Apagado\"\n};\n\n// Enviar el resultado al
siguiente bloque\nreturn msg;\n",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 260,
        "y": 600,
```

```
    "wires": [
        [
            "8ba7e946245dd1df"
        ]
    ]
},
{
    "id": "8e7cf5f6b06e09aa",
    "type": "mqtt-broker",
    "name": "MQTT-ESP32",
    "broker": "broker.emqx.io",
    "port": 1883,
    "clientid": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closeQos": "0",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
```

```
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```